A REPORT

ON

3D HUMAN POSE ESTIMATION FOR CRICKET BROADCASTING

Multi-camera pose for broadcast stories and Unreal Engine animation

BY

Name of the student	ID No.	Discipline
Aksh Shah	2024A7PS0532G	B.E. Computer Science

Prepared in partial fulfilment of the
Practice School - I Course

AT

Quidich Innovation Labs, Mumbai

A Practice School - I

Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

(K. K. Birla Goa Campus)

June, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Practice School Division

Station: Quidich Innovation Labs | Centre: Remote (Online)

Duration: 8 weeks | Date of start: 25 May 2026

Date of submission: 20 June 2026

Title of the project: 3D Human Pose Estimation for Cricket Broadcasting (multi-camera pose for broadcast stories and Unreal Engine animation).

ID No., Name and Discipline of the student: 2024A7PS0532G, Aksh Shah, B.E. Computer Science.

Name and Designation of the expert: Ms. Simone Singh, Mentor, Quidich Innovation Labs.

Name of the PS faculty member: Prof. Tarkeshwar Singh, Associate Professor, Department of Mathematics, BITS Pilani K. K. Birla Goa Campus.

Key words: human pose estimation, multi-camera triangulation, re-identification, cricket broadcast analytics, model benchmarking.

Project areas: computer vision, sports broadcast technology, applied machine learning.

Abstract: This report documents the first half of a Practice School project at Quidich Innovation Labs on real-time 3D human pose estimation for cricket broadcast. The first two weeks established the technical foundations of 2D and 3D pose estimation and produced an audited survey of more than fifty pose models released after 2020, framed around the speed against accuracy trade-off that governs live broadcast use. The third week turned that survey into a reproducible benchmarking repository with a single measurement protocol. The fourth week applied this foundation to a real seven-camera calibrated cricket capture: a foundation-readiness audit (Phase 0) and a per-camera person detection and 2D pose stage (Phase 1) were completed and verified on a full delivery. The report closes with a quantified account of the remaining work for the second half, beginning with per-camera tracking and progressing to cross-camera re-identification, role assignment, and smooth 3D reconstruction for downstream story creation and Unreal Engine rendering.

Signature of the student | Signature of the PS faculty member

Date: 20 June 2026 | Date:

Acknowledgements

I thank Quidich Innovation Labs for the opportunity to work on a live sports broadcast problem and for access to the calibrated multi-camera cricket data and the existing ball-tracking pipeline that this project builds upon. I am grateful to my company mentor, Ms. Simone Singh, for technical guidance and for framing the problem in production terms.
I thank my Practice School faculty in-charge, Prof. Tarkeshwar Singh, and the BITS Pilani Practice School Division for their supervision and feedback during the first half of the programme. I also acknowledge my project teammates for the collaborative discussions that shaped the group's shared design, while noting that the implementation and analysis presented in this individual report are my own contribution.

Signature of the student
Date: 20 June 2026

Contents

1 Introduction **9**
- 1.1 Organisation and project context . . . 9
- 1.2 Why body pose matters: events and stories . . . 9
- 1.3 Team structure and individual scope . . . 10
- 1.4 The capture setup and the core challenge . . . 11

2 Literature Review **12**
- 2.1 From 2D images to 3D pose . . . 12
- 2.2 The speed against accuracy trade-off . . . 12
- 2.3 How the metrics are read . . . 13
- 2.4 The surveyed models . . . 14

3 Methodology **15**
- 3.1 The benchmarking repository (Week 3) . . . 15
- 3.2 Group 1 pipeline design (Week 4) . . . 16
- 3.3 Phase 0: foundation readiness . . . 17
- 3.4 Phase 1: per-camera perception . . . 17

4 Results and Discussions **18**
- 4.1 The cricket dataset . . . 18
- 4.2 Phase 0 result: technically complete, externally blocked . . . 18
- 4.3 Phase 1 result: per-camera detection and 2D pose . . . 19
- 4.4 Supporting evidence: public-dataset benchmark . . . 20
- 4.5 The honest gap . . . 20

5 Conclusions and Future Work **22**
- 5.1 Summary of the first four weeks . . . 22
- 5.2 Phase 2: per-camera tracking (immediate next work) . . . 22
- 5.3 Roadmap: Phases 3 to 7 . . . 23
- 5.4 The smoothness problem (Phase 6 preview) . . . 23
- 5.5 Concluding remarks . . . 24

6 Recommendations **25**

Bibliography **28**

Appendices **29**

A Group 1 output contract schema **30**

B Extended model comparison **31**

List of Figures

1.1 The broadcast value chain. Calibrated cameras feed Group 1, which produces identified 3D pose. That pose, combined with match events, drives broadcast stories and is also rendered as an Unreal Engine animation. 10

2.1 The route from 2D pose in each camera to a smooth 3D animation. Triangulation combines calibrated views into 3D joints, which are then cleaned and smoothed before export. 12
2.2 Speed against accuracy for representative pose models. Positions are an illustrative summary of the audited survey, not exact benchmark coordinates; the underlying numbers are in Tables 2.2 and 2.3 and Appendix B. 13

3.1 The five-stage benchmarking pipeline. Configuration files are the source of truth, and every run is recorded with full metadata so results are comparable across people and machines. 15
3.2 The Group 1 pipeline as eight dependency-ordered phases. Phases 0 and 1 (green) were completed in Week 4; Phase 2 (amber) is the immediate next step. 16
3.3 The existing single-point ball pipeline (top) and the Group 1 extension to many players and many joints (bottom). The processing shape is reused; the data is generalised. 16

4.1 The seven-camera rig and shared world-coordinate convention from the project's calibration reference. The cameras ring the pitch and point inward; all are calibrated into a single origin and axis system, which lets a joint seen in several cameras be triangulated into one 3D point. 19
4.2 Phase 0 outcome. All technical checks pass; the remaining items are escalated management decisions, not code gaps. The output contract is frozen so Groups 2 and 3 can proceed in parallel. 19
4.3 Total player detections per camera over 600 frames. The field view (camera04) sees the most players and the tight side view (camera07) the fewest, consistent with the rig geometry. 20
4.4 The honest gap after Phase 1. Detection works, but the output is jittery, identity-less, and role-less. Each limitation maps to the later phase that resolves it. 21

5.1 Per-camera tracking turns relabelled per-frame detections into stable tracklets, each following one player and bridging brief occlusion. 22

5.2 The planned staged cleanup pipeline. Raw, jittery, anonymous detections enter and one clean, identified, smooth 3D skeleton leaves, ready for broadcast [3, 17]. 24

List of Tables

1.1 Representative broadcast stories that require human body pose. Source: the project's internal story catalogue. 10
1.2 The three project groups and their dependencies. 10

2.1 Plain-language glossary of the pose metrics used in this report. 13
2.2 Body-centric, real-time pose candidates (condensed from the audited survey). 14
2.3 Dense whole-body pose candidates (condensed from the audited survey). . . 14

3.1 The four checks performed by the Phase 0 readiness audit. 17

4.1 The Phase 0 verified properties of the supplied cricket capture. 18
4.2 Verified Phase 1 run statistics (run p1-yolo26x, full delivery, all 7 cameras). . 20

5.1 Roadmap of the remaining Group 1 phases. 23
5.2 Noise sources in multi-view 3D reconstruction and the stage that removes each. 24

6.1 Use-case-driven model recommendations from the survey. 25

B.1 Extended comparison of representative post-2020 pose models. 31

Chapter 1

Introduction

1.1 Organisation and project context

Quidich Innovation Labs is a sports broadcast technology company. It builds the systems that turn raw match footage into the graphics and analytical segments that viewers see during a live cricket broadcast. One established product line is Decision Review System (DRS) style ball tracking, in which several calibrated cameras observe a single moving ball, its position is reconstructed in three dimensions, and the reconstruction is rendered as a broadcast graphic. This Practice School project extends that established idea from a single ball point to full human players.

The eventual goal of the project is **real-time three dimensional (3D) human pose estimation** for cricket broadcast. A *pose* is the set of positions of a person's body joints, such as the shoulders, elbows, knees, and ankles. A *2D pose* gives those joints on the flat image plane of one camera. A *3D pose* gives them in real world space, which is what is needed to drive broadcast graphics and biomechanical analysis. The output of the work is intended to be consumed in two forms: a structured data file describing each player's joints over time, and a clean, rigged animation that can be rendered inside Unreal Engine, the game engine used for broadcast visualisation.

1.2 Why body pose matters: events and stories

A cricket broadcast is organised around *events* and *stories*. An event is a single passage of play, for example a delivery that results in a wide, a no-ball, a run-out, or a wicket. A story is a short explanatory or analytical segment built around an event. Many of the most valuable stories cannot be told from ball tracking alone, because they depend on where a player's body is. Table 1.1 lists representative examples taken from the project's story catalogue, each of which requires body pose rather than the ball alone.

These stories explain why accurate, stable body pose is a foundational capability for the broadcaster. The pose data feeds two distinct downstream consumers: an analytics layer that composes the stories, and a rendering layer that animates the player in Unreal Engine. Figure 1.1 shows this value chain.

Table 1.1: Representative broadcast stories that require human body pose. Source: the project's internal story catalogue.

Story	Key body landmarks	What the pose adds
Waist-high full toss (no-ball)	Batter waist, shoulders, head	Estimates where the batter's waist would be standing upright and checks whether the ball passed above it.
Front-foot no-ball	Bowler front foot	Pinpoints, frame by frame, exactly where the bowler's front foot lands relative to the popping crease.
Wide ball decision	Batter feet, hips, shoulder, lead arm	Tracks how far the batter can reach and whether the batter moved toward or away from the ball.
Stumping or run-out	Batter feet, bat tip	Locates the foot and bat at the instant the bails are dislodged to judge whether the ground was made in time.
LBW impact distance	Batter feet, pads	Measures how far from the stumps the ball struck the pads, which affects the reliability of the decision.
Bowler release mechanics	Wrist, shoulder, jump foot, stride	Reveals the body mechanics behind extra pace or a disguised slower ball.

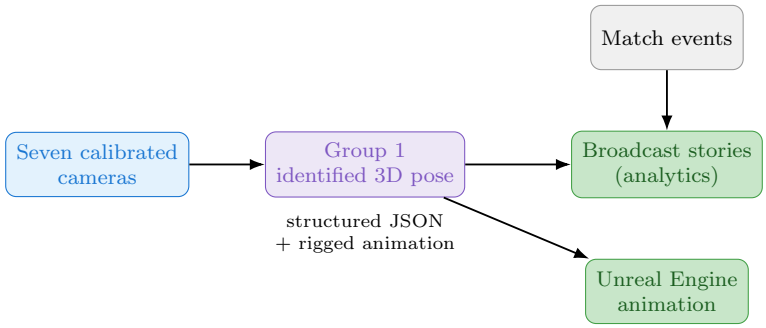

Figure 1.1: The broadcast value chain. Calibrated cameras feed Group 1, which produces identified 3D pose. That pose, combined with match events, drives broadcast stories and is also rendered as an Unreal Engine animation.

1.3 Team structure and individual scope

The broader project is divided across three groups whose work is interdependent, summarised in Table 1.2. Group 1 is the perception and geometry layer: it turns camera footage into identified, role-labelled, smooth 3D skeletons. Group 2 composes the broadcast stories, and Group 3 interprets the cricket event. Groups 2 and 3 both depend on Group 1's output.

Table 1.2: The three project groups and their dependencies.

Group	Responsibility	Relationship
Group 1	Player re-identification, role assignment, multi-camera tracking, and clean 3D pose.	Supplies pose data to Groups 2 and 3 and animation to Unreal Engine.
Group 2	Story creation.	Consumes Group 1 pose and Group 3 events.
Group 3	Interpretation of the complete cricket event.	Supplies event understanding to Group 2.

This report concerns my individual work within Group 1. Group 1 work is necessarily collaborative, and where group context is needed it is described accurately, but the audit, benchmarking, implementation, and analysis presented here as completed work are my own contribution. The remaining phases are distributed across the team; the specific ownership of future phases is stated in Chapter 5 only to make the plan concrete.

1.4 The capture setup and the core challenge

The cricket data is produced by a rig of seven synchronised, calibrated cameras arranged around the pitch, the same class of rig Quidich uses for ball tracking. Because each camera is calibrated into one shared world coordinate system, a joint that is seen by several cameras can be combined into a single 3D point by a geometric procedure called *triangulation*.

The central engineering challenge is not simply detecting poses in one image. It is preserving each player's identity across all seven views and over time, and producing 3D motion that is stable and smooth enough to render on broadcast, despite noise, occlusion, missing views, and geometric error. Chapters 2 to 4 describe how the first four weeks built toward this goal, and Chapter 5 sets out the remaining work.

Chapter 2

Literature Review

The first two weeks were a structured study of human pose estimation and a survey of the available models, motivated by a single practical question: which pose estimation model should the project adopt. This chapter records that study and the audited survey it produced.

2.1 From 2D images to 3D pose

A single camera produces a flat image and therefore cannot, on its own, recover true depth. Two image points that lie on the same line of sight project to the same pixel, so depth is ambiguous from one view. The standard solution is multiple calibrated cameras [6]. When the same joint is detected in two or more calibrated views, the lines of sight from each camera intersect at a single point in space, and that intersection is the 3D position of the joint. This is triangulation. With more cameras the estimate becomes more robust, because a joint hidden in one view may still be visible in others. Figure 2.1 shows the path from per-camera 2D joints to a clean 3D animation.

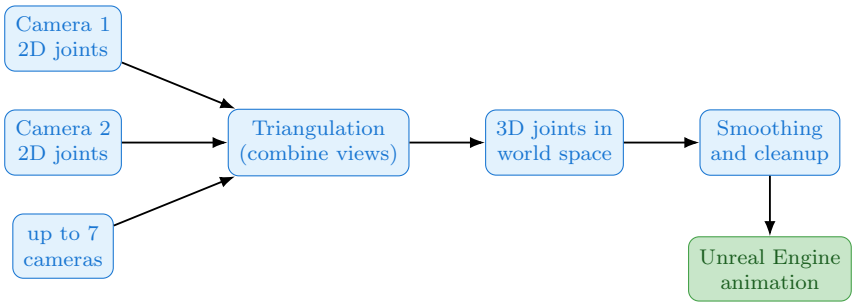

Figure 2.1: The route from 2D pose in each camera to a smooth 3D animation. Triangulation combines calibrated views into 3D joints, which are then cleaned and smoothed before export.

2.2 The speed against accuracy trade-off

Pose models occupy a spectrum between two extremes. At one end are large, highly accurate models such as Meta's Sapiens family [12], which produce detailed skeletons but run far too slowly for live broadcast. At the other end are small models that run at hundreds of frames

per second but miss joints, particularly when players overlap or are partly hidden. Live cricket requires neither extreme. It requires a middle ground: a model fast enough for real-time use yet accurate enough that the reconstructed 3D skeleton is correct. Figure 2.2 positions representative models on this trade-off and marks the region of interest.

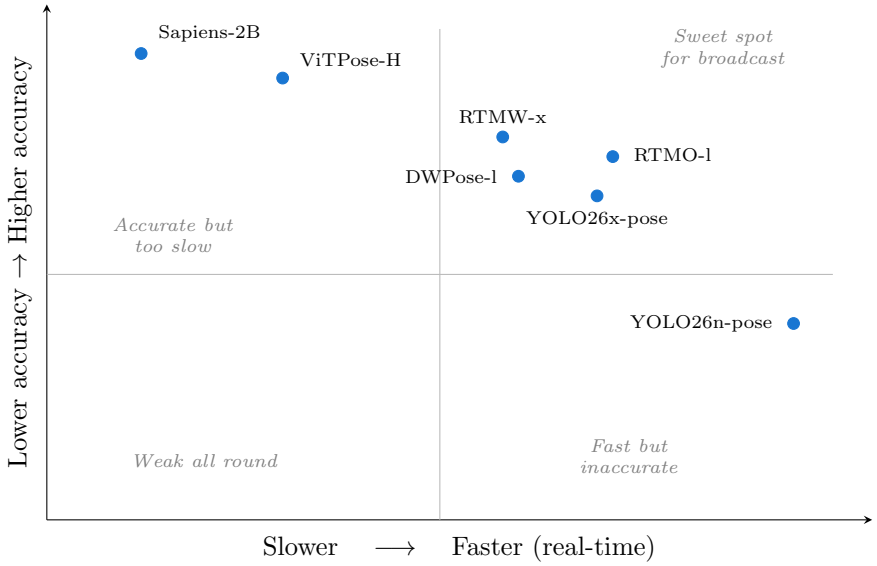

Figure 2.2: Speed against accuracy for representative pose models. Positions are an illustrative summary of the audited survey, not exact benchmark coordinates; the underlying numbers are in Tables 2.2 and 2.3 and Appendix B.

2.3 How the metrics are read

Comparing models requires a shared vocabulary of metrics. Table 2.1 defines the ones used throughout this report in plain terms. A recurring caveat documented during the survey is that these numbers are only comparable when models are tested on the *same dataset and hardware*. For example, the 17-joint COCO benchmark [13], the 133-joint COCO-WholeBody benchmark [9], and the separate 308-joint test used by the Sapiens v2 family are different protocols, so their scores must not be compared head to head.

Table 2.1: Plain-language glossary of the pose metrics used in this report.

Term	**Meaning**	**Why it matters**
AP (AP50-95)	Average Precision: how often predicted joints land close to the truth, averaged over strict to loose tolerances. Higher is better.	Standard accuracy score for 2D pose.
MPJPE	Mean Per Joint Position Error, in millimetres: average distance between a predicted 3D joint and its true location. Lower is better.	Standard accuracy score for 3D pose.
FPS	Frames per second the model can process.	Decides whether it can run live.
Latency (ms)	Time to process one frame. Lower is better.	The frame-by-frame real-time constraint.
Params / GFLOPs	Model size and compute per frame.	Proxy for hardware cost and deployability.

2.4 The surveyed models

The survey catalogued more than fifty model variants across two families: dense *whole-body* models that estimate the body together with hands, feet, and face, and lighter *body-only* models that estimate the seventeen main joints. Every figure in the audited tables is backed by an official paper, repository, or model card, with conflicting published numbers flagged rather than silently averaged. This audit was consolidated into two internal production-focused review documents that underpin the conclusions in this chapter [18, 19]. Tables 2.2 and 2.3 present a verified, condensed slice; the extended table is in Appendix B.

Table 2.2: Body-centric, real-time pose candidates (condensed from the audited survey).

Model	**Accuracy (COCO AP)**	**Speed**	**Size**	**Why it is of interest**
YOLO26x-pose [10]	71.6 AP	about 82 FPS on T4 (12.2 ms)	57.6M params	Newest YOLO pose head; broad export to ONNX, TensorRT, CoreML.
RTMO-l [14]	74.8 AP; CrowdPose-Hard 65.3	141 FPS on V100	44.8M params	One-stage; cost does not grow with player count; strong under occlusion.
RTMPose-l [7]	75.8 AP	TensorRT-FP16 3.46 ms per crop	27.7M params	Very fast per person; mobile friendly.
ViTPose-H [22]	79.1 AP	241 FPS on A100 (batch 64)	632M params	Accuracy ceiling for 2D body; heavy.

Table 2.3: Dense whole-body pose candidates (condensed from the audited survey).

Model	**Accuracy (Whole-body AP)**	**Size**	**Why it is of interest**
DWPose-l [23]	66.5 (384x288)	about 4.5M params	Trained to infer hidden joints; good under occlusion.
RTMW-x [8]	70.2 (384x288)	mid-size	Best balance of whole-body detail and speed.
Sapiens-2B [12]	74.4	2.16B params	Highest open whole-body accuracy; offline only.

The clearest conclusion from the survey is that dense whole-body models substantially improve suitability for cricket, because they capture the wrists at ball release and the feet for no-ball calls, but the strongest dense models give up real-time speed unless the pipeline is carefully staged. The body-only family, led by RTMO and the YOLO pose heads [15], offers practical real-time operation at the cost of finer whole-body detail. A recent systematic review of real-time pose estimation reaches a consistent conclusion about this staging trade-off [4]. These findings directly informed the model recommendations in Chapter 6 and the baseline choice in Chapter 3.

Chapter 3

Methodology

This chapter describes the methodology of the third and fourth weeks: first the benchmarking repository that makes all later experiments comparable, then the Group 1 pipeline design and the methodology of the two phases completed on the real cricket data.

3.1 The benchmarking repository (Week 3)

The survey produced numbers collected from published sources. To compare any model the team tests in future, those tests must be run the same way, on the same metrics, with full record of the hardware and software used. Without a fixed protocol, two people testing two models would produce numbers that cannot be compared. The third week therefore turned the survey into a benchmarking repository: shared code and a single fixed protocol so that every later result is reproducible and directly comparable.

The repository is driven from configuration files that act as the single source of truth for which models, datasets, metrics, and skeletons exist. A single command line tool runs a five-stage pipeline, shown in Figure 3.1: `prepare` fetches and checks data, `smoke` runs a one-image readiness check per model, `run` executes the benchmark, `aggregate` collates the per-run outputs, and `report` produces the comparison tables. Each run writes a small, timestamped folder containing its metrics together with hardware and software manifests, so that two contributors never conflict and every number is traceable to the exact conditions that produced it. To avoid drawing speed conclusions from incompatible setups, each model runs in its own isolated software environment, and the repository records that laptops are for functional validation rather than speed conclusions.

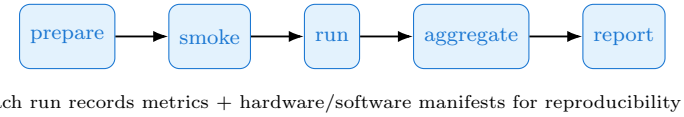

Figure 3.1: The five-stage benchmarking pipeline. Configuration files are the source of truth, and every run is recorded with full metadata so results are comparable across people and machines.

3.2 Group 1 pipeline design (Week 4)

With real cricket data available, the Group 1 task was decomposed into eight phases, P0 to P7, ordered by dependency rather than by calendar. Figure 3.2 shows the full pipeline and marks the two phases completed in Week 4. The guiding design principle is *geometry first*: because both teams wear near-identical kits, appearance alone cannot reliably tell players apart, so the calibrated camera geometry is used as the primary signal for association and 3D reconstruction.

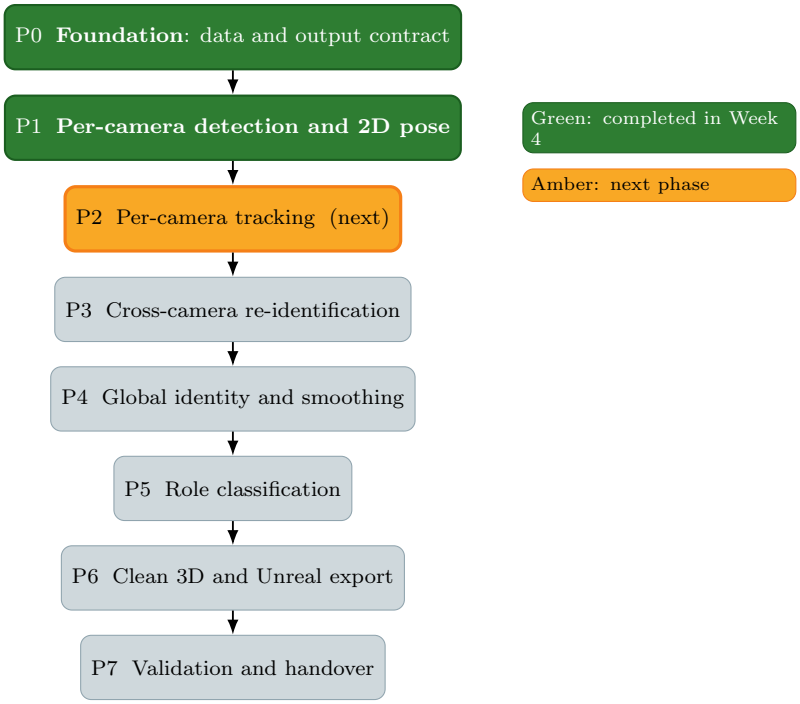

Figure 3.2: The Group 1 pipeline as eight dependency-ordered phases. Phases 0 and 1 (green) were completed in Week 4; Phase 2 (amber) is the immediate next step.

A second design decision is to reuse the existing ball pipeline. That pipeline already detects a single point per camera, triangulates it to 3D, cleans and trims the track, and exports it to Unreal. The shape of this chain is exactly what Group 1 needs; the difference is that Group 1 must handle many people and many joints rather than a single point. Figure 3.3 shows this correspondence.

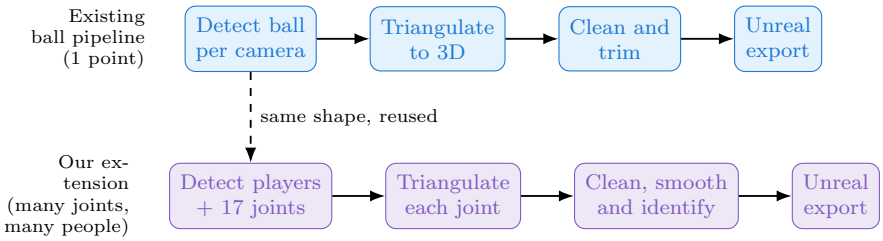

Figure 3.3: The existing single-point ball pipeline (top) and the Group 1 extension to many players and many joints (bottom). The processing shape is reused; the data is generalised.

3.3 Phase 0: foundation readiness

The purpose of Phase 0 is to prove the ground is solid before any model code is written: that the data is complete and synchronised, that the calibration files load and behave correctly, and that the output format handed to Groups 2 and 3 is agreed and frozen. Skipping this step risks a missing frame, a broken calibration file, or a vague handoff format surfacing weeks later as a confusing bug.

The method is an automated readiness audit that performs the four checks in Table 3.1, each of which writes a compact evidence file. The audit deliberately separates two questions: whether the engineering is sound, and whether the management-level decisions that sit outside the code (for example who owns the validation data and what accuracy counts as passing) have been resolved.

Table 3.1: The four checks performed by the Phase 0 readiness audit.

Check	**What it proves**
Dataset inventory	All 7 cameras present, 600 frames each, frame identifiers synchronised, correct 2560x1440 resolution.
Calibration	Calibration files load, matrices are valid, surveyed points project correctly, and the existing ball reprojection is reproducible.
Events pipeline	The existing ball artifact chain is present and its reprojection errors are summarised.
Output contract	The JSON format to be delivered to Groups 2 and 3 is explicit and validates in code.

The output contract deserves emphasis. It fixes the structure of the data each downstream group receives, with the identity, role, and 3D fields present as placeholders that later phases fill. Freezing the contract early means Groups 2 and 3 can build against it in parallel, before the later phases are complete. The full schema is given in Appendix A.

3.4 Phase 1: per-camera perception

The purpose of Phase 1 is, for every camera and every frame, to find each person as a bounding box and to estimate that person's 2D pose as seventeen body joints with a confidence score for each. Three terms are used: a *bounding box* is a rectangle around a detected person; the *2D pose* is the on-image position of the seventeen standard COCO joints (eyes, shoulders, elbows, wrists, hips, knees, ankles, and so on); and *confidence* is how sure the model is about each joint, which later stages use to reject noise. The existing detector finds only the ball, so detecting multiple players and their joints is net-new work and the front end on which every later phase depends.

The method is a per-camera person detector and 2D pose stage built on YOLO26x-pose [10], run over the full delivery. This model was chosen first deliberately, following the Week 1 to 2 shortlist: it is the fastest path to a working baseline, combining detection and pose in a single model with easy GPU deployment and broad export. It is the minimum-viable baseline, not the final choice. The run used a CUDA GPU at image size 640, batch size 32, half precision (FP16), full-frame inference, with non-maximum-suppression IoU 0.7 and a detection confidence threshold of 0.25. The results are reported in Chapter 4.

Chapter 4

Results and Discussions

This chapter reports the verified outcomes of the two completed phases on the real cricket dataset, together with supporting evidence from the public-dataset benchmark, and then states honestly the gap that the remaining phases must close.

4.1 The cricket dataset

Phase 0 confirmed the structure of the supplied capture, summarised in Table 4.1. The seven cameras are calibrated into one shared world coordinate system, which is what makes triangulation possible. Figure 4.1 shows the rig layout and coordinate convention from the project's calibration reference.

Table 4.1: The Phase 0 verified properties of the supplied cricket capture.

Item	Detail
Cameras	7 synchronised, calibrated cameras (camera01 to camera07) in three capture groups.
Footage	600 frames per camera per delivery; the worked delivery is 4,200 frames across the 7 cameras.
Resolution	2560 x 1440 pixels per frame.
Calibration	Bundle-adjusted intrinsics and extrinsics, per-camera 3x4 projection matrices, and a pitch-plane configuration.
Existing pipeline	A working ball-tracking chain that detects, triangulates, cleans, and exports a single point to Unreal.

4.2 Phase 0 result: technically complete, externally blocked

The audit produced a clear and intentionally honest split. The internal status is a pass: all technical checks are green, the calibration is usable, and the output contract validates in code. The external status is blocked, with seven open items that are management decisions rather than code gaps, for example the ownership of the validation dataset, who produces the manual ground-truth labels, and what accuracy thresholds count as passing. Figure 4.2 shows this split. These items were surfaced and escalated rather than guessed, which is the correct engineering posture: the engineering side of Phase 0 is complete, and the remaining items require a decision from the company.

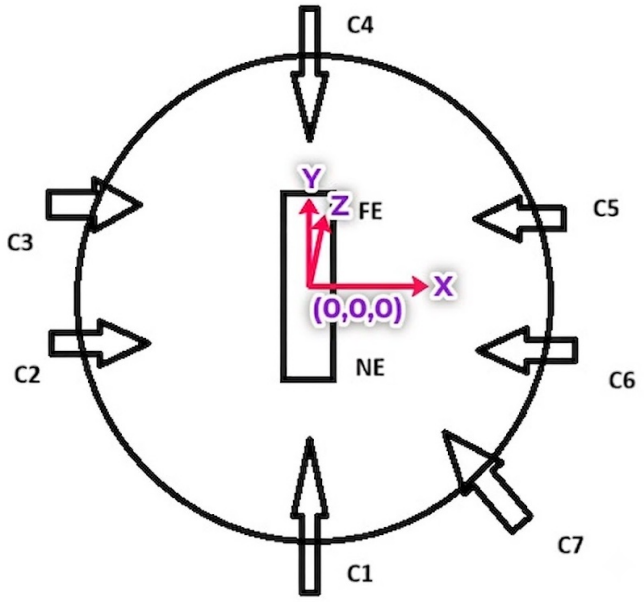

Figure 4.1: The seven-camera rig and shared world-coordinate convention from the project's calibration reference. The cameras ring the pitch and point inward; all are calibrated into a single origin and axis system, which lets a joint seen in several cameras be triangulated into one 3D point.

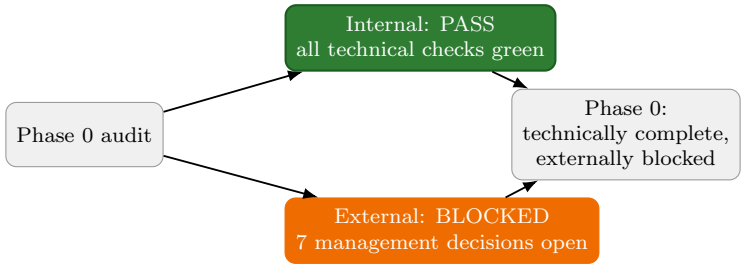

Figure 4.2: Phase 0 outcome. All technical checks pass; the remaining items are escalated management decisions, not code gaps. The output contract is frozen so Groups 2 and 3 can proceed in parallel.

4.3 Phase 1 result: per-camera detection and 2D pose

Phase 1 produced per-camera person detections and 2D poses over the full delivery. The verified run statistics are in Table 4.2. Across 4,200 frame records the stage produced 13,170 player detections with no empty or failed frames, at a median model inference time of 11.3 ms per frame.

The number of detections per camera reflects how many players each view actually sees, shown in Figure 4.3. The end-on bowler and field cameras see the most players, while the tight side view (camera07) sees the fewest. The average number of players seen per frame ranged from 5.0 in the field view down to 1.1 in the tight view, which is consistent with the camera layout in Figure 4.1.

Table 4.2: Verified Phase 1 run statistics (run p1-yolo26x, full delivery, all 7 cameras).

Metric	Value
Frame records written	4,200 (600 x 7 cameras)
Total player detections	13,170
Median model inference	11.3 ms per frame (p95: 20.7 ms)
End-to-end speed (offline batch)	16.6 FPS (including image decode and disk I/O)
Empty or failed frames	0
Status	pass

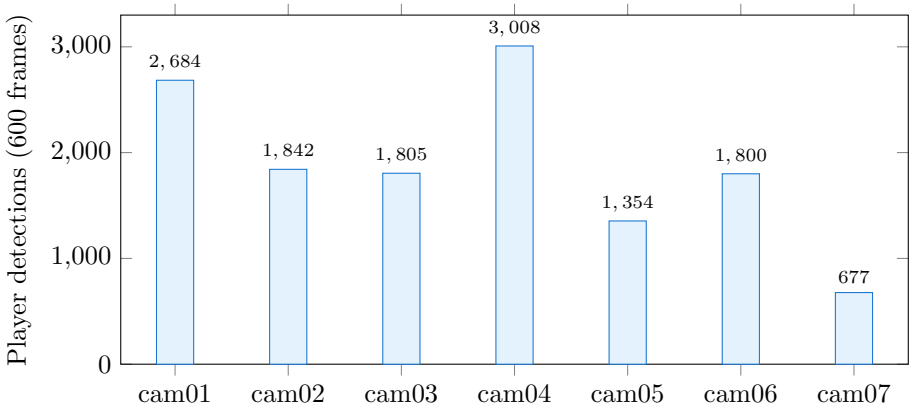

Figure 4.3: Total player detections per camera over 600 frames. The field view (camera04) sees the most players and the tight side view (camera07) the fewest, consistent with the rig geometry.

4.4 Supporting evidence: public-dataset benchmark

To anchor the chosen baseline against a standard reference, the same model was benchmarked through the Week 3 repository on the full 5,000-image COCO 2017 keypoint validation set [13]. YOLO26x-pose scored an OKS Average Precision of 0.710 (AP50 0.904, AP75 0.779) with an average recall of 0.771, at a median latency of 42.6 ms per image (batch size 8, image size 640, single-precision). This run is recorded with full hardware and software metadata in the repository. It is reported here as supporting evidence for the baseline's accuracy on a public benchmark, and it is deliberately not compared directly against the cricket run, because the protocol, precision, and batch settings differ.

4.5 The honest gap

The Phase 1 detections are correct, but watching the rendered overlay video makes three limitations obvious, and these are exactly what the later phases exist to fix. First, the skeletons jitter from frame to frame, because each frame is processed independently with no temporal memory. Second, there are no identities: the same player is treated as a new detection in every frame and in every camera. Third, there are no roles: the system does not know who is the bowler, the striker, or the wicketkeeper. Figure 4.4 maps each limitation to the phase that resolves it. This gap is the launch point for the second half of the project.

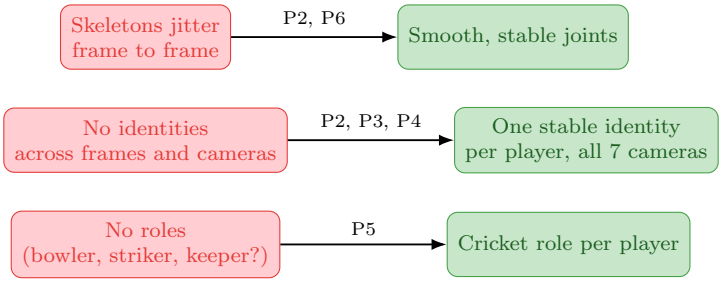

Figure 4.4: The honest gap after Phase 1. Detection works, but the output is jittery, identity-less, and role-less. Each limitation maps to the later phase that resolves it.

Chapter 5

Conclusions and Future Work

5.1 Summary of the first four weeks

The first four weeks established a solid, verified foundation. Weeks 1 and 2 framed the core speed against accuracy trade-off, surveyed and audited more than fifty post-2020 pose models with sourced numbers, and established the multi-camera triangulation path from 2D to 3D. Week 3 turned that survey into a reproducible benchmarking repository with a single measurement protocol. Week 4 applied this foundation to real cricket data: Phase 0 passed all technical readiness checks and froze the output contract, and Phase 1 produced verified per-camera detection and 2D pose over a full delivery, with 13,170 detections across 4,200 frames and no failures. The work that remains is to turn these correct but jittery, anonymous detections into clean, identified, smooth 3D motion.

5.2 Phase 2: per-camera tracking (immediate next work)

The immediate next phase gives each camera a memory of players over time. Phase 1 finds people from scratch in every frame and has no notion that a person in one frame is the same person in the next. Phase 2 links a player's detections across time within a single camera, so that the same person keeps one stable local track identifier through movement and brief occlusion. The output is a set of per-camera tracklets, where a tracklet is one person's continuous path through that camera's frames. Figure 5.1 shows the intended effect.

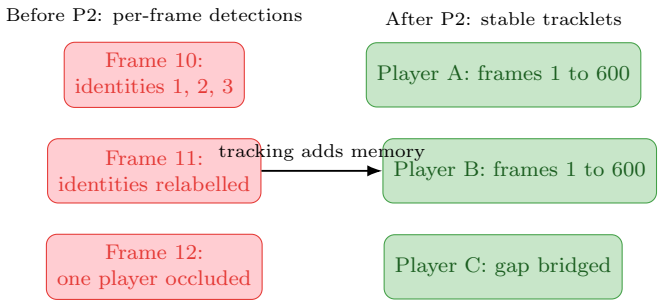

Figure 5.1: Per-camera tracking turns relabelled per-frame detections into stable tracklets, each following one player and bridging brief occlusion.

The method is standard multi-object tracking adapted to the footage, matching each

detection to an existing track using three signals: a Kalman motion model [11] that predicts where a tracked player should next appear; a compact appearance embedding to confirm matches, noting that identical kits limit how much appearance can contribute; and a geometry prior from the calibrated cameras to gate matches by real-world plausibility. Several trackers will be benchmarked on the project's own data through the Week 3 protocol, including DeepSORT [21] and the ByteTrack and BoT-SORT class [24, 1], alongside a geometry-assisted variant. The reported measures will be the number of identity switches per delivery within a camera (lower is better) and track completeness (the fraction of a player's true path that is followed). The exit criterion is stable per-camera tracklets for a full delivery, with those numbers recorded; this becomes the input to cross-camera re-identification in Phase 3.

5.3 Roadmap: Phases 3 to 7

The remaining phases combine the seven cameras, attach identities and roles, and produce smooth 3D for Unreal. They are distributed across the team as set out in Table 5.1, which states ownership only to make the plan concrete.

Table 5.1: Roadmap of the remaining Group 1 phases.

Phase	In one sentence	Owner
P3	Match the same physical player across all 7 views using geometry first (triangulation and reprojection, epipolar lines, and a ground-plane foot test), since kits are identical.	Teammate
P4	Collapse per-camera tracks and cross-camera matches into one stable global identity per player for the whole delivery, repairing identity switches and bridging occlusion.	Teammate
P5	Label each player with a cricket role (bowler, striker, non-striker, wicketkeeper, umpire, fielder) using pitch geometry, motion rules, and role priors.	Teammate
P6	Triangulate the joints into 3D and run a multi-stage cleanup so the skeleton is smooth enough to animate, then export to JSON and to Unreal Engine.	Teammate
P7	Measure association accuracy, identity switches, and role accuracy on a blind dataset, and deliver the final reports and handover.	Teammate

The key idea in Phase 3 is to lead with geometry rather than appearance. Because both teams wear near-identical kits, appearance-based matching is unreliable. Instead, if a player seen in one camera and a player seen in another triangulate to the same point in the real world, and their feet land on the same spot on the pitch, they are the same person. This reuses the exact reprojection check already proven in the ball pipeline.

5.4 The smoothness problem (Phase 6 preview)

The hardest part of the whole mandate is making the 3D skeleton smooth enough for Unreal. Any residual jitter appears as a twitching limb on broadcast. Multi-view triangulation is inherently jittery: one noisy joint, one missing camera, or one bad line of sight can make a 3D joint jump between frames. The planned answer is a staged cleanup pipeline in which each known noise source is removed by a specific stage, as mapped in Table 5.2 and sequenced in Figure 5.2.

Table 5.2: Noise sources in multi-view 3D reconstruction and the stage that removes each.

Noise source	**Removed by**
2D keypoint jitter	A. Confidence gating and D. Temporal smoothing
Triangulation outliers (one bad ray)	A. RANSAC ray rejection [5] and C. Reprojection-error rejection
Missing views or occlusion	B. Confidence-weighted triangulation and F. inverse kinematics fill
Reprojection error (calibration slack)	C. Reprojection-error rejection
Bone-length drift	E. Skeleton constraints (constant bone length)
Foot-skate (feet sliding)	F. Inverse kinematics and foot-lock
Rotation jitter	G. Quaternion SLERP rotation smoothing [20]

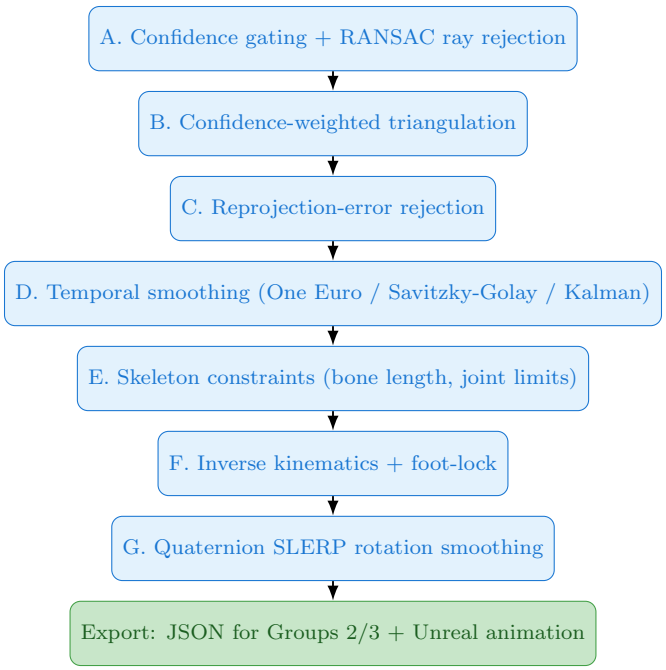

Figure 5.2: The planned staged cleanup pipeline. Raw, jittery, anonymous detections enter and one clean, identified, smooth 3D skeleton leaves, ready for broadcast [3, 17].

5.5 Concluding remarks

The first half of the Practice School delivered a verified foundation rather than a premature end to end demonstration: an audited model survey, a reproducible benchmarking framework, a passing foundation audit, and a working per-camera perception stage on real cricket data. The honest assessment is that the detections are correct but not yet stable, identified, or three dimensional. The plan for the second half is concrete and dependency-ordered, beginning with per-camera tracking and progressing through geometry-first cross-camera association, role assignment, and staged 3D cleanup toward the smooth, identified output that the broadcast stories and Unreal Engine rendering require.

Chapter 6

Recommendations

The following recommendations follow from the audited survey and the Week 4 results.

Match the model to the use case. No single pose model is best for every job. The survey supports the use-case-driven shortlist in Table 6.1: a whole-body model for balanced cricket biomechanics, an occlusion-robust whole-body model for heavily obscured joints, a one-stage body model for real-time multi-player tracking, and a fast, broadly exportable model for a first deployment. The Week 4 baseline deliberately started from the last of these and should be revisited against the others once tracking and association are in place.

Table 6.1: Use-case-driven model recommendations from the survey.

Use case	**Recommended model**	**Reason**
Balanced overall	RTMW-x / RTMW-l	Whole-body detail plus practical speed; fits cricket biomechanics.
Heavy occlusion	DWPose-l	Explicitly trained to infer hidden joints (bat, pads, gloves).
Real-time, many players	RTMO-l	One-stage; cost independent of player count; strong under crowding.
Fast first deployment	YOLO26x-pose	Quick to ship, broad export support, good enough accuracy.

Lead with geometry, not appearance. Because the two teams wear near-identical kits, cross-camera matching and 3D cleanup should rely first on the calibrated camera geometry, with appearance used only as a tie-breaker. This reuses the proven ball-pipeline reprojection check and is more robust than appearance-only association.

Keep every experiment reproducible. All future model and tracker comparisons should run through the Week 3 benchmarking protocol, with full hardware and software metadata recorded, so that results remain comparable across people and machines. Speed conclusions should be drawn only from appropriate GPU hardware, not from laptops.

Escalate the open management decisions early. The remaining Phase 0 blockers, namely ground-truth ownership, annotation tooling, and the validation accuracy targets, are decisions rather than code. Resolving them early is necessary before the validation in Phase

7 can produce meaningful numbers, and obtaining additional match data would strengthen any generalisation claims, since the current work uses a single match.

Bibliography

[1] Nir Aharon, Roy Orfaig, and Ben-Zion Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.

[2] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang, and Matthias Grundmann. BlazePose: On-device real-time body pose tracking. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.

[3] Géry Casiez, Nicolas Roussel, and Daniel Vogel. 1 euro filter: A simple speed-based low-pass filter for noisy input in interactive systems. In *ACM Conference on Human Factors in Computing Systems (CHI)*, 2012.

[4] Yingying Chen, Zhenan Feng, Daniel Paes, Daniel Nilsson, and Ruggiero Lovreglio. Real-time human pose estimation and tracking on monocular videos: A systematic literature review. *Neurocomputing*, 655:131309, 2025.

[5] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[6] Richard Hartley and Andrew Zisserman. Multiple view geometry in computer vision. 2004.

[7] Tao Jiang, Peng Lu, Li Zhang, Ningsheng Ma, Rui Han, Chengqi Lyu, Yining Li, and Kai Chen. RTMPose: Real-time multi-person pose estimation based on MMPose. *arXiv preprint arXiv:2303.07399*, 2023.

[8] Tao Jiang, Xinchen Xie, and Yining Li. RTMW: Real-time multi-person 2d and 3d whole-body pose estimation. *arXiv preprint arXiv:2407.08634*, 2024.

[9] Sheng Jin, Lumin Xu, Jin Xu, Can Wang, Wentao Liu, Chen Qian, Wanli Ouyang, and Ping Luo. Whole-body human pose estimation in the wild. In *European Conference on Computer Vision (ECCV)*, 2020.

[10] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO pose estimation (yolo11 and yolo26). https://docs.ultralytics.com/tasks/pose, 2025.

[11] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[12] Rawal Khirodkar, Timur Bagautdinov, Julieta Martinez, Su Zhaoen, Austin James, Peter Selednik, Stuart Anderson, and Shunsuke Saito. Sapiens: Foundation for human vision models. *arXiv preprint arXiv:2408.12569*, 2024.

[13] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[14] Peng Lu, Tao Jiang, Yining Li, Xiangtai Li, Kai Chen, and Wenming Yang. RTMO: Towards high-performance one-stage real-time multi-person pose estimation. *arXiv preprint arXiv:2312.07526*, 2023.

[15] Debapriya Maji, Soyeb Nagori, Manu Mathew, and Deepak Poddar. YOLO-Pose: Enhancing YOLO for multi-person pose estimation using object keypoint similarity loss. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022.

[16] Meta AI Research. Sapiens2: Foundation human vision models (source repository and model collection). https://github.com/facebookresearch/sapiens2, 2026.

[17] Abraham Savitzky and Marcel J. E. Golay. Smoothing and differentiation of data by simplified least squares procedures. *Analytical Chemistry*, 36(8):1627–1639, 1964.

[18] Aksh Shah. Production-centric deep research review of real-time human pose estimation for sports analytics and cricket. Technical report, Quidich Innovation Labs (internal project review), 2026.

[19] Aksh Shah. Production-focused comparative analysis of human pose estimation models for sports analytics and cricket. Technical report, Quidich Innovation Labs (internal project review), 2026.

[20] Ken Shoemake. Animating rotation with quaternion curves. In *ACM SIGGRAPH Computer Graphics*, 1985.

[21] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric (DeepSORT). In *IEEE International Conference on Image Processing (ICIP)*, 2017.

[22] Yufei Xu, Jing Zhang, Qiming Zhang, and Dacheng Tao. ViTPose: Simple vision transformer baselines for human pose estimation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[23] Zhendong Yang, Ailing Zeng, Chun Yuan, and Yu Li. Effective whole-body pose estimation with two-stages distillation (DWPose). *arXiv preprint arXiv:2307.15880*, 2023.

[24] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022.

Appendices

Appendix A

Group 1 output contract schema

The frozen output contract delivered to Groups 2 and 3 uses the schema identifier `g1_player_frame/v0` with the COCO-17 skeleton. The identity, role, and 3D fields are present as placeholders that later phases fill, which lets the downstream groups build against the format in parallel.

```
{
  "camera_id": "cam_01",
  "frame_index": 212334,
  "players": [{
    "bbox_xywh_px": [x, y, w, h],          // filled in Phase 1
    "pose_2d": {
      "keypoints_px": [ ... 17 ],          // filled in Phase 1
      "confidence":   [ ... 17 ]           // filled in Phase 1
    },
    "global_player_id": null,              // filled in Phase 4
    "role": "unknown",                     // filled in Phase 5
    "pose_3d": null                        // filled in Phase 6
  }]
}
```

The role field takes one of the values bowler, striker, non_striker, wicketkeeper, umpire, fielder, or unknown.

Appendix B

Extended model comparison

Table B.1 gives a wider slice of the audited survey. All figures are drawn from official papers, repositories, or model cards, and accuracy figures are only comparable within the same benchmark protocol (COCO-17, COCO-WholeBody-133, or the Sapiens v2 308-keypoint test).

Table B.1: Extended comparison of representative post-2020 pose models.

Model	Type	Accuracy	Size	Notes
RTMPose-m [7]	2D body	74.6 COCO AP	13.6M	430+ FPS on GTX 1660 Ti.
RTMPose-l [7]	2D body	75.8 COCO AP	27.7M	TensorRT-FP16 3.46 ms per crop.
RTMO-m [14]	2D body, one-stage	70.9 COCO AP	22.6M	about 81 FPS on V100.
RTMO-l [14]	2D body, one-stage	74.8 AP; CrowdPose-Hard 65.3	44.8M	141 FPS on V100; strong occlusion behaviour.
ViTPose-B [22]	2D body, transformer	75.8 COCO AP	86M	944 FPS on A100 (batch 64).
ViTPose-H [22]	2D body, transformer	79.1 COCO AP	632M	Accuracy ceiling for 2D body.
DWPose-l [23]	2D whole-body	66.5 Whole AP (384x288)	about 4.5M	Distilled to infer hidden joints.
RTMW-l [8]	2D whole-body	70.1 Whole AP (384x288)	mid-size	Whole-body detail with practical speed.
RTMW-x [8]	2D whole-body	70.2 Whole AP (384x288)	mid-size	Best whole-body balance in the survey.
Sapiens-1B [12]	2D dense whole-body	72.7 Whole AP	1.17B	Near-offline high-resolution analytics.
Sapiens-2B [12]	2D dense whole-body	74.4 Whole AP	2.16B	Highest open whole-body accuracy.
Sapiens2-1B [16]	2D dense (308 kpt)	80.4 mAP (308-kpt test)	1.46B	Separate 308-keypoint protocol; not comparable to COCO.
YOLO11x-pose [10]	2D body	69.5 COCO AP	58.8M	82.6 FPS on T4.
YOLO26x-pose [10]	2D body	71.6 COCO AP	57.6M	Week 4 baseline; broad export support.
BlazePose Heavy [2]	2D + relative 3D	task-specific PCK	about 26 MB	On-device, single-person fitness tracking.